

Package the Spring Boot Student API into a Docker container so it can run identically on any machine. Then push the image to Docker Hub (public registry) for deployment. Docker is the industry-standard for modern deployment.

- **Docker Desktop** – <https://www.docker.com/products/docker-desktop> (free)
- **Docker Hub account** – <https://hub.docker.com> (free signup)

```
# In terminal, inside the project folder:
mvn clean package -DskipTests

# This creates: target/student-api-0.0.1-SNAPSHOT.jar
# That JAR file contains your entire Spring Boot application
```

[illegible]

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Step 3 – Create .dockerignore

Create **.dockerignore** in the project root:

```
target/  
.git/  
*.md  
.idea/  
*.iml
```

Step 4 – Build the Docker Image

Replace 'yourusername' with your actual Docker Hub username

```
docker build -t yourusername/student-api:1.0 .
```

Verify image was created:

```
docker images
```

Expected output:

# REPOSITORY	TAG	IMAGE ID	SIZE
# yourusername/student-api	1.0	abc123def	180MB

Step 5 – Run the Container Locally

Run the container (maps container port 8080 to host port 8080)

```
docker run -p 8080:8080 --name student-api-container yourusername/student-api:1.0
```

To run in background (detached mode):

```
docker run -d -p 8080:8080 --name student-api-container yourusername/student-api:1.0
```

Check it's running:

```
docker ps
```

Test the API (open browser or Postman):

<http://localhost:8080/api/students>

Stop the container:

```
docker stop student-api-container
```

Step 6 – Create docker-compose.yml (with MySQL)

Docker Compose runs multiple containers together. This runs both the app and MySQL:

docker-compose.yml (in project root)

```
version: '3.8'
```

```
services:
```

```
  # MySQL database container
```

```
  mysql:
```

```
    image: mysql:8.0
```

```
    container_name: mysql-db
```

```
    environment:
```

```
      MYSQL_ROOT_PASSWORD: root123
```

```
      MYSQL_DATABASE: studentdb
```

```
      MYSQL_USER: intern
```

```
      MYSQL_PASSWORD: intern123
```

```
    ports:
```

```
      - "3306:3306"
```

```
    volumes:
```

```
      - mysql-data:/var/lib/mysql
```

```

healthcheck:
  test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
  interval: 10s
  timeout: 5s
  retries: 5

# Spring Boot app container
app:
  build: .
  container_name: student-api
  depends_on:
    mysql:
      condition: service_healthy
  environment:
    SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/studentdb
    SPRING_DATASOURCE_USERNAME: intern
    SPRING_DATASOURCE_PASSWORD: intern123
  ports:
    - "8080:8080"

volumes:
  mysql-data:

```

Step 7 – Run with Docker Compose

```

# Start all containers:
docker compose up --build

# Run in background:
docker compose up -d --build

# View logs:
docker compose logs -f app

# Stop everything:
docker compose down

# Stop and delete volumes (reset database):
docker compose down -v

```

Step 8 – Push Image to Docker Hub

```

# Login to Docker Hub:
docker login

# Tag the image (if not already tagged):
docker tag yourusername/student-api:1.0 yourusername/student-api:latest

# Push to Docker Hub:
docker push yourusername/student-api:1.0
docker push yourusername/student-api:latest

# Your image is now public at:
# https://hub.docker.com/r/yourusername/student-api

```

■ Expected Outcome

- Docker image builds successfully: docker build completes with no errors
- Container runs: docker run starts app, API accessible at localhost:8080
- docker ps shows running container

- Docker Compose: both mysql and app containers start together
- Image pushed to Docker Hub – public URL available

■ **GitHub Upload Checklist**

- Folder: task4-dockerize-deploy/
- Dockerfile, .dockerignore, docker-compose.yml
- Screenshot of docker ps showing running container
- Screenshot of API working inside Docker (Postman or browser)
- Docker Hub image URL (e.g. <https://hub.docker.com/r/vasavi/student-api>)
- Include this PDF